长上下文技术横纵分析报告

> 从 512 tokens 到 10M tokens，一场关于"模型能看多远"的技术长征。

一、一句话定义

长上下文技术（Long Context Technology）是一套让大语言模型**处理远超训练长度的文本序列**的技术体系——包括位置编码扩展、注意力计算优化、KV Cache 压缩和分布式序列并行。它的核心矛盾是：如何在不显著增加算力成本的前提下，让数百页文档的内容都被模型"看在眼里"。

> 🎯 **读完这篇你能**：理清位置编码扩展、注意力计算优化、KV Cache 压缩三条技术路线，能判断为什么有些模型支持 1M token 而有些止步于 128K。

二、技术背景

核心矛盾：N^2 的诅咒

Transformer 的 self-attention 计算量随序列长度 N 呈平方级增长——N 翻倍，计算量翻 4 倍。一个 128K 序列的 attention 计算量是 4K 序列的 **1024 倍**。这种暴力增长让"把文本变长"在数学上就极其昂贵。

三条技术阵线

围绕这个矛盾，长上下文技术从三个方向同时突破：

方向	要解决什么	关键进展	
1	位置编码	模型怎么"理解"超过训练长度的位置	RoPE→YaRN/NTK→LongRoPE
2	计算优化	$O(N^2)$ 的 attention 怎么算得动	FlashAttention→Ring Attention
3	显存压缩	KV Cache 怎么装得下	MQA→GQA→MLA

三条线缺一不可。位置编码决定了"能不能推到那么远"，计算优化决定了"能不能算得动"，显存压缩决定了"能装下多大的缓存"。本报告的核心脉络就是这三条线如何交叉演进，最终把上下文从 2017 年的 512 tokens 推到 2026 年的 10M tokens。

> 前置知识请参考独立文档 《长上下文技术_前置知识》，涵盖位置编码、KV Cache、FlashAttention 等基础概念的通俗讲解。

> 📚 **关联报告**

> – Transformer — 长上下文的核心矛盾 $O(N^2)$ 来自 Transformer 的 self-attention

> – 大模型推理部署 — KV Cache 压缩是推理部署的核心难题，直接影响长上下文可用性

> – Context Engineering — 上下文窗口扩大后，如何设计上下文本身变成一门工程学科

> – RAG — 长上下文和 RAG 是解决"模型知识外部化"的两条互补路线

三、纵向分析：从诞生到当下

3.1 原点：$O(N^2)$ 的诅咒（2017）

2017 年 6 月，Google 的 Vaswani 等人发表了《Attention Is All You Need》，Transformer 架构诞生。没有人预料到这篇论文会在未来十年统治整个 NLP 领域，但它的一个"隐含代价"从一开始就刻在基因里。

Self-attention 的时间复杂度和空间复杂度都是 $O(N^2)$。当序列长度为 512 时，这个平方项还不算致命。但当序列长度达到 128K 时，它会把 GPU 显存吃得干干净净。

2017-2019 年，几乎所有 Transformer 模型的上下文长度都锁死在 512-1024 tokens——不是不想做长，是算力和显存不允许。

3.2 早期突围：稀疏与近似（2019-2020）

这个阶段的探索者试图用一种古老但有效的思路解决 $O(N^2)$：**不计算所有注意力对，只算一部分。**

Sparse Transformer（OpenAI, 2019.04） 率先打破僵局。Child、Gray、Radford 和 Sutskever 提出将注意力矩阵稀疏分解，把复杂度降到 $O(N\sqrt{N})$。在 Enwik8 基准上，他们首次让 Transformer 建模了数万个时间步长的序列。

紧接着是 **Reformer（Google, 2020.01）**——用局部敏感哈希（LSH）近似注意力，把复杂度降到 $O(L \log L)$。同时引入可逆残差层，中间激活值不用全存，反向传播时重算即可。Reformer 在长序列上的内存效率比 Transformer 高出一个数量级。

2020 年 4 月和 7 月，**Longformer**（Allen AI）和 **BigBird**（Google）相继出场。两者都走了"局部滑动窗口 + 少量全局 token"的路线——大部分注意力只在局部窗口内计算，少数关键位置（如 [CLS] token）拥有全局视野。

这些早期探索的共性：**都在试图绕开 O(N^2)，方法是让注意力变"稀疏"或变"近似"。** 但绕开的代价是模型质量下降——稀疏注意力无法建模某些需要全局依赖关系的任务。Longformer 和 BigBird 的"局部+全局"混合方案是当时最实用的折中，但终究不是 full attention 的等价物。

2020 年底的 **Long Range Arena（LRA）** 基准测试系统性地揭了底：10 种高效 Transformer 没有一种能在所有任务上匹敌原始 full attention 的质量。**"近似"带来了效率，但带走了精度。** 这个矛盾，要到 FlashAttention 的出现才算真正解决。

3.3 两条路：位置编码的分岔口（2021）

如果 attention 计算效率暂时解决不了，另一个方向是：**让模型在训练长度之外也能正常运作。** 这就是位置编码的外推问题。

2021 年是位置编码的转折年。两条关键路线在这一年几乎同时出现：

RoPE（苏剑林团队, 2021.04） 提出了一种优雅的方案：不把位置信息加到 embedding 上，而是旋转 query 和 key 向量。旋转角度取决于位置——两个 token 越近，它们的旋转量之差越小，attention 分数越高。从群论角度看，RoPE 让位置编码和语义编码在几何上正交：旋转不改变向量模长（语义由模长承载），角度差编码相对距离。这种解耦是数学层面的优雅。

RoPE 还附带一个关键性质：**query 在位置 m 和 key 在位置 n 的点积，只依赖于它们的相对距离 m−n**。由此，模型学到的是"相对位置关系"，理论上天然支持外推。

ALiBi（Meta, 2021.08） 走了完全不同的路。Press、Smith 和 Lewis 干脆去掉位置编码，直接在 attention 分数上加一个与距离成比例的负偏置——距离越远，扣分越多。训练比标准 Transformer 快 11%、省 11% 内存，而且天然支持外推到训练长度的 2 倍以上。

2021 年，RoPE 和 ALiBi 都能工作，没人能断定谁会赢。但回头来看，RoPE 的"可扩展性"成了决定性优势——它不像 ALiBi 那样有固定的偏置曲线，后来的研究者可以通过调整旋转频率（YaRN、NTK scaling 等方法）把上下文推到任何长度。这个"可调性"在长上下文竞赛中成了生死攸关的能力。

3.4 FlashAttention：工程突破（2022）

2022 年 5 月，斯坦福的 Tri Dao 发表了 FlashAttention。这篇论文没有发明新的注意力变体，而是在系统层面做了一个极其聪明的优化。

标准 attention 的低效不在计算量本身，而在**显存带宽**。Attention 需要反复读写一个巨大的 N×N 矩阵到 HBM（高带宽显存），HBM 的带宽远低于 GPU 的 SRAM 高速缓存。用论文里的比喻：**计算本身不是瓶颈，把数据搬来搬去才是。**

FlashAttention 的核心是 **tiling（分块计算）**——把注意力矩阵切成小块，在 SRAM 里一块一块算完，不把完整矩阵写回 HBM。显存读写从 O(N^2) 降到 O(N)。

这个"看起来只是工程优化"的工作，实际效果是革命性的：它让 128K+ 序列的训练在工程上**首次可行**。

FlashAttention 2（2023.07）进一步优化了并行策略，达到 A100 理论计算峰值的 50–73%。FlashAttention 3（2024）利用 Hopper 架构的 FP8 Tensor Core 和异步指令，再提速 1.5–2×。

FlashAttention 的历史地位很特殊：它没有发明"更好的注意力"，而是让"已有的 full attention"跑得更快。 但它对行业格局的影响，可能超过任何一篇注意力变体的论文——因为它是所有后续长上下文模型的基础设施。

3.5 上下文元年：军备竞赛拉开（2023）

2023 年是长上下文技术从实验室走向产业的转折年。几条线索在这一年交汇。

3.5.1 模型发布潮

5 月，Anthropic 发布了 Claude 100K——这是第一个主流商用模型把上下文推到 100K。一个普通用户可以把整本《了不起的盖茨比》一次性丢给模型分析。硅谷震动。

10 月，Anthropic 再次加码，Claude 200K 发布——四倍于 GPT-4（当时 32K）。

11 月，OpenAI 在 DevDay 上反击，GPT-4 Turbo 支持 128K。但更重要的是，这个发布揭示了一个格局变化：**上下文长度已经成为模型竞争的核心参数，和推理能力并列。**

12 月，Google Gemini 1.0 发布，但只支持 32K。所有人都知道 Google 在憋大招。

3.5.2 位置编码扩展成熟

2023 年，位置编码扩展方法密集发表，RoPE 的"可扩展性"红利开始兑现。

Position Interpolation（PI, Meta, 2023.06） 提出一个简单的思路：线性缩小位置索引。例如要把 128K 映射到 4K 的训练窗口，只需把所有位置索引除以 32。理论上，插值的上界比外推小 600 倍，稳定性大幅提升。仅需 1000 步微调就能把 LLaMA 从 2K 扩展到 32K。

YaRN（2023.08） 更进一步：不压缩位置索引，而是调整 RoPE 的旋转频率。高频维度（负责精细定位）用保守缩放，低频维度（负责粗略定位）用激进缩放。效果比 PI 好 10 倍——只需 PI 十分之一的 token 就能达到同等效果。

同期，**NTK-aware scaling** 从社区中涌现（Reddit 讨论 → 论文论述），方法论类似但更早出现。Code LLaMA（2023.08）直接用 NTK-aware scaling 把上下文从 16K 训练扩展到 100K 推理。

这两条路线（PI → YaRN，社区 NTK）最终合并为行业标准做法：**RoPE + 分频缩放 + 渐进式继续训练**。几乎所有后续模型都沿用了这个配方。

3.5.3 KV Cache 压缩的现实需求

上下文变长后，另一个问题浮出水面：推理时 KV Cache 的显存占用。

MQA（Multi-Query Attention, Google, 2019）最早提出所有 query head 共享一组 K/V。但质量问题让它没有广泛采用。

GQA（Grouped Query Attention, Google, 2023.05） 是 MQA 和 MHA（标准多头注意力）的折中：query heads 分成若干组，组内共享 K/V。Llama 2 70B 率先采用（8/32 KV heads 比例），Mistral 跟进，最终成为 Llama 3、Gemma 等模型的标配。

GQA 不是革命性创新，但它的"折中哲学"很关键——在实际部署中，**"够好"比"完美"更重要**。

3.5.4 分布式序列并行

单卡放不下长序列怎么办？分到多卡。

DeepSpeed Ulysses（微软, 2023.09） 基于 All-to-All 通信实现序列并行——序列长度和计算设备数等比例增长时，通信量恒定。

Ring Attention（UC Berkeley, 2023.10） 更优雅：多设备组成通信环，每个设备计算 attention 的一块，分块结果在环上流转。通信与计算重叠，理论上支持"近无限上下文"。

两者的本质都是**把 N 维度的压力分布到多卡**，而不是试图在一张卡上硬扛。

3.5.5 评估方法的诞生

2023 年 11 月，Greg Kamradt 发布了 **Needle in a Haystack（大海捞针）** 评测方法——在长文本中藏一句事实，问模型能不能找到。这个至今仍是长上下文评测的行业标杆。

它的历史价值在于：**第一次让用户直观地看到"宣称上下文"和"有效上下文"的差距。** 一个模型可能宣称 128K 上下文，但 NIAH 测试显示它在 64K 以后就开始"漏针"。

3.6 百万级时代：从 100K 到 1M（2024）

如果说 2023 年是 100K 级别的竞赛，2024 年则直接跳到百万级。

3.6.1 Gemini 1.5 Pro 的降维打击

2024 年 2 月，Google 发布 Gemini 1.5 Pro。默认支持 1M token，研究预览版支持 10M。更重要的是，它在 NIAH 测试中 1M 长度上 recall 超过 99.7%。

这个结果让整个行业重新评估 Google 的长上下文能力。Google 在基础设施（TPU）、系统优化（技术细节未公开，但已知使用了自研的位置编码增强）上的积累，一次性地在长上下文上释放出来。

Claude 3.5 Sonnet（200K, 2024.06） 和 **GPT-4o（128K, 2024.05）** 守住各自阵地。Anthropic 没跟进百万级——事后看来，可能是策略选择（聚焦安全性 > 上下文竞赛）。

3.6.2 Llama 3.1：开源的 128K

2024 年 7 月，Meta 发布 Llama 3.1 405B，直接支持 128K 上下文。技术报告揭示了一个关键细节：Llama 3.1 将 RoPE base frequency 从 10000 改为 500000——一个简单的参数调整，让模型在预训练阶段就"为长上下文做好准备"。

从此，长上下文能力不再是闭源模型的专利。开源社区迅速跟进：**Qwen 2（32K→128K）、DeepSeek-V2（128K）、Yi（200K）、GLM-4（128K）**——2024 年上半年，128K 成为模型的"基础配置"。

3.6.3 MLA：DeepSeek 的 KV Cache 革命

2024 年 5 月，DeepSeek-V2 发布了 **Multi-Head Latent Attention (MLA)**。

MLA 的核心：不直接存储 K/V，而是投影到低维 latent 空间压缩存储，推理时再解压。KV Cache 降低 93.3%。

想象一个图书馆：传统方案是把每本书的全文放在前台（MHA），GQA 是把一个书架放一本代表书，MLA 则是把每本书压缩成"摘要卡片"——占空间极小，需要时再还原。

MLA 的革命性在于：**它不像 MQA/GQA 那样通过牺牲表达力来节省显存，而是通过信息压缩来同时达到"省显存+保质量"**。DeepSeek-V2 的基准测试表明，MLA 在大多数任务上匹敌甚至超越标准 MHA。

从商业角度看，MLA 是 DeepSeek 最聪明的技术投资之一。它让 DeepSeek 可以用更少的硬件资源支撑极长的上下文——在 2024-2025 年的价格战中，这是巨大的成本优势。

3.6.4 评估的升级：RULER 和 BABILong

行业很快发现 NIAH 的局限性——找"明文密码"太简单，不能反映真实的语义理解能力。

RULER（NVIDIA, 2024.04） 扩展了 NIAH 的变体：多针检索、多类型检索、多跳追踪、聚合任务。17 个模型在 13 个任务上的评估显示：尽管在标准 NIAH 上接近完美，几乎所有模型在更长序列上的表现大幅下降，仅一半能在 32K 长

度保持满意性能。

BABILong（NeurIPS 2024） 更残酷：20 种推理任务（事实链式推理、归纳、演绎、计数等），发现主流 LLM 有效利用的仅 10–20% 的上下文，性能随推理复杂度急剧下降。

中文长上下文评估方面，**CLongEval（EMNLP 2024）** 填补了空白——7 个任务、7267 个样本、1K 到 100K 范围，系统评估了 6 个开源和 2 个商业中文模型。

至此，"评估体系"完成了从简单（NIAH）到复杂（RULER）到综合（BABILong）的跃迁。

3.7 技术的成熟与考验（2025–2026）

3.7.1 1M 成为标配

2025–2026 年，上下文竞赛进入"饱和期"：

时间	事件	上下文
2025.02	Gemini 2.5 Pro	1M+（百万级准确率 >99%）
2025.06	Claude 4 (Opus/Sonnet)	200K
2025.12	Gemini 2.0 Flash	1M
2026	Llama 4 Scout	**10M**（行业最大）
2026	GPT–5.5	1M（opt–in）
2026	DeepSeek V4 Pro	1M
2026	Claude Opus 4.6/4.7	1M

2026 年 4 月：**1M token 已不再是差异化因素**——至少是旗舰模型的"标配"。

3.7.2 Llama 4 Scout 的 10M 与争议

Llama 4 Scout 宣称 10M 上下文，是行业最高。但 NoLiMa 评测（2025.12）暴露了真相：Scout 的有效上下文仅约 1K——在 32K tokens（容量的 0.3%）时，语义检索性能已经崩溃到 21.6%。

Scout 使用了 **iRoPE（Interleaved RoPE）** 技术：3 层 RoPE 后跟 1 层 NoPE（无位置编码），NoPE 层创造"信息高速公路"让远距离 token 通信。但代价是位置精度——捕获了"大意"，失去了"细节"。

这是一个典型的 trade–off：**10M 的"宣称"靠牺牲精度换来的，在需要精确检索的任务上几乎不可用。** Scout 更适合"找大致相关段落"的检索场景，不适合"精确回答某个事实"的推理场景。

3.7.3 DeepSeek V4 Pro：成本效率的新标杆

DeepSeek V4 Pro 延续了 MLA 的优势，在 1M 上下文下，单 token 推理 FLOPs 仅为 V3.2 的 27%，KV Cache 仅为 10%。384K 的输出长度是行业平均的 3–6 倍。

同时，DeepSeek 的 API 定价（$1.74/M input, $0.145/M cache）远低于 Claude（$5/$0.50）和 GPT–5.5（$5/$2.50）。长上下文的高成本一直被认为是商业化瓶颈，DeepSeek 正在打破这个瓶颈。

3.7.4 NoLiMa 基准：揭穿"虚假上下文"

2025 年 12 月的 NoLiMa 评测是整个行业的一盆冷水。它用无词汇重叠的 needle 测试语义检索，结果：

模型	宣称上下文	有效上下文（NoLiMa）	@32K 退化
Llama 4 Scout	10M	~1K	−73.6%
GPT-4.1	1M	~16K	−17.7%
GPT-4o	128K	~8K	−29.8%
Gemini 2.0 Flash	1M	~4K	−54.1%
Claude 3.5 Sonnet	200K	~4K	−66.0%

所有模型都"注水"了，只是程度不同。 字面检索（NIAH）基本全满分，但语义检索（NoLiMa）暴露了真实差距。"Lost in the Middle" 在 2026 年依然是未解决的难题——attention 权重必须在更多 token 间分配，注意力自然稀释。

3.8 三条技术线的终极汇合

回顾从 2017 到 2026 的九年代际变迁，长上下文技术的演进不是单一突破的结果，而是三条并行技术线的最终汇合：

线一——位置编码：Absolute (2017) → ALiBi vs RoPE (2021) → RoPE 胜出 → PI/YaRN/NTK (2023) → iRoPE/LongRoPE (2025)。这条线的本质是"让模型理解没见过的位置"。

线二——计算效率：Sparse/LSH (2019–2020) → FlashAttention (2022) → FA2/FA3 → Ring Attention (2023)。这条线的本质是"让 $O(N^2)$ 算得快、算得起"。

线三——显存压缩：MQA (2019) → GQA (2023) → MLA (2024)。KVQuant (2024) 将 KV Cache 量化到 INT3 使 1M 上下文在单卡上可行。这条线的本质是"让缓存装得下"。

2026 年的状态：三条线各自成熟后，汇合成一个完整的技术栈。位置编码让模型"看到"长距离，FlashAttention 让长序列"算得动"，MLA/KVQuant 让 KV Cache "装得下"——三者缺一不可。

四、横向分析：竞争图谱

4.1 场景判断：竞品充分（场景 C）

长上下文技术不是单一产品，而是贯穿模型厂商、开源社区、推理框架、评估体系的多层赛道。选取最具代表性的竞争者进行对比。

4.2 旗舰模型上下文全景

模型	发布	宣称上下文	有效上下文（NoLiMa）	位置编码	关键技术	开源
Gemini 2.5 Pro	2025-02	1M	~1M (NIAH)	RoPE（增强）	自研 3D 位置建模	❌
Claude Opus 4.6/4.7	2025-2026	1M	~900K	RoPE 变体	Prompt Caching 90%	❌
GPT-5.5	2026	1M	~16K (NoLiMa)	RoPE 自研	272K 标准，1M opt-in	❌
DeepSeek V4 Pro	2026	1M	未公开	RoPE + MLA	KV Cache 降至 10%	❌
Llama 4 Scout	2026	10M	~1K (NoLiMa)	iRoPE	Interleaved NoPE 层	✅
Llama 3.1 405B	2024-07	128K	~8K (推算)	RoPE (base=500k)	YaRN 扩展	✅
Qwen 2.5	2024	128K-1M	未公开	RoPE	dual chunk attention	✅
Mistral	2024	32K-128K	未公开	RoPE	滑动窗口	✅
Yi-34B	2024-01	200K	未公开	RoPE + NTK + YaRN	三种扩展组合	✅

4.3 技术路线深度对比

4.3.1 位置编码：RoPE 的绝对统治

2021 年 ALiBi 和 RoPE 还是旗鼓相当的对手。到 2026 年，ROPE 几乎统治了所有主流模型。

ALiBi 的致命弱点在于它的"固定偏置曲线"——它对所有 token 施加相同的距离惩罚，无法通过后期调整来扩展上下文。而 RoPE 的旋转频率是可调的，PI、YaRN、NTK 等方法形成了一个完整的扩展工具生态。

这个分岔点的历史意义重大：ALiBi 是"设计时就定死"，RoPE 是"留了扩展接口"。在长上下文竞赛中，有扩展接口的路线最终胜出。

4.3.2 KV Cache 压缩：三条路线

方案	压缩原理	KV Cache 节省	质量损失	采用者	成熟度
MQA	所有 query 共享 K/V	1/H（~95%）	较大	PaLM, Falcon	历史方案
GQA	query 分组共享 K/V	1–1/H（~75%）	轻微	Llama 2/3, Mistral, Gemma	当前主流
MLA	低秩 latent 压缩	~93–98%	极小	DeepSeek 全系列	最佳实践

GQA 的流行不是因为最好，而是因为**"足够好 + 实现简单"**。Llama 3 选择 GQA 而非 MLA，反映了 Meta 对实现复杂度的考量——GQA 是架构层面的一行配置，MLA 需要全新的 attention 实现。

MLA 是技术角度明显更优的方案，但实现门槛更高。 2026 年的趋势是，更多模型可能跟随 DeepSeek 采用 MLA 或类似方案。

4.3.3 Attention 计算：FlashAttention 的事实标准

所有支持长上下文的模型都依赖 FlashAttention。GPT-4、Claude、Gemini、Llama、DeepSeek——没有例外。

FlashAttention 的路线：

版本	年份	加速比	硬件要求	核心改进
FA1	2022	baseline	A100	tiling 分块，O(N) HBM 访问
FA2	2023	2× FA1	A100	优化并行，50–73% 理论峰值
FA3	2024	1.5–2× FA2	H100/H200	FP8 + 异步指令

4.3.4 推理优化：框架层面的竞争

框架	核心技术	适用场景	优势
vLLM	PagedAttention	通用推理	KV Cache 碎片消除，吞吐最高
SGLang	RadixAttention	共享前缀场景	前缀缓存，树状结构推理
TensorRT-LLM	内联融合、INT4/FP8	NVIDIA 生态	单卡推理最优

4.4 成本对比：长上下文的经济学

模型	输入 $/1M tokens	输出 $/1M tokens	缓存 $/1M tokens
Claude Opus 4.6	$5.00	$25.00	$0.50（90% 缓存命中）
Gemini 3.1 Pro	$2.00–$4.00	$12.00	未公开
GPT-5.5	$5.00	未公开	$2.50（batch）
DeepSeek V4 Pro	$1.74	未公开	$0.145
GPT-5-Mini	$0.25	$2.00	未公开
DeepSeek V3.2	$0.28	$0.42	未公开

Claude Opus 的 output 定价 ($25/M) 是 DeepSeek V3.2 ($0.42) 的 60 倍。**长上下文场景下，模型选择对成本的影响是数量级的。**

Prompt Caching 是 2026 年最关键的成本杠杆——Claude 的 90% 缓存折扣意味着重复性长上下文任务的 token 成本降至十分之一。DeepSeek 的缓存定价 ($0.145/M) 是所有方案中最低的。

4.5 长上下文 vs RAG：2026 年的共识

2025–2026 年主流观点：**不是"二选一"，而是混合架构。**

维度	长上下文胜出	RAG 胜出
单文档问答	+34%	—
跨文档综合	—	+67%
成本	—	8× 更低
动态数据	—	显著更优
延迟	—	快得多

决策框架：

- **需要全局理解 + 文档不长（<200K）**：长上下文直出
- **文档超大（>1M）+ 需要精确检索**：RAG 召回 → 长上下文分析
- **需要外部知识更新**：RAG 必须
- **高频调用 + 成本敏感**：RAG 优先

2026 年的实践趋势是 **Context Engineering**：用 RAG 召回相关 chunk，用长上下文模型做综合处理。这不是"退步"，而是对两种工具各自优势的务实认识。

4.6 用户口碑与社区评价

开发者普遍焦虑的是定价问题。 Reddit r/LLMDevs 上反复出现的讨论：一次使用 200K+ token 调用旗舰模型（Claude/GPT-4）的成本可达 $5-20，这对产品化是一个真实障碍。

对模型能力的评价分两极。 字面检索场景（"从这篇文档找出这句话"）几乎所有 1M 模型都表现不错。但语义理解场景（"总结第三部分的核心论点"）在超过 100K 后质量显著下降。Reddit r/singularity 上对 Gemini 的称赞集中在 NIAH 表现，而 r/LocalLLaMA 社区的讨论更务实——"我的 32K 本地模型已经覆盖了 95% 的日常需求"。

一个有趣的现象：大多数开发者承认长上下文"很酷"，但实际产品中使用超过 32K 的场景不到 20%。**巨量上下文更像"保险"而非"日常"**——你知道它在那里，但很少真正用到极限。

五、横纵交汇洞察

5.1 历史的路径锁定：RoPE 获胜不是因为它最好

回顾纵向发展，RoPE 的"胜利"不完全是因为技术最优，而是因为**它在正确的时间点提供了正确的接口**。

2021 年，当 RoPE 和 ALiBi 还是竞争对手时，没有人能预见 2023 年的上下文竞赛。但 RoPE 的旋转频率参数（base frequency）提供了一种"后期扩展能力"——2023 年的 PI、YaRN、NTK 等方法全部建立在 RoPE 的可调性之上。ALiBi 虽然本身外推能力更好，但它没有"旋钮"可以拧。

这个案例揭示了一个更深层的规律：**在快速演化的领域，"可扩展性"和"可调整性"有时候比"当前性能"更重要。** RoPE 赢的不是 2021，而是 2023—2026。

5.2 有效上下文的"水分"：竞赛催生的营销泡沫

纵向历史显示一个清晰的 pattern：**每次模型宣称上下文大幅提升，都会伴随着"有效上下文"指标的滞后验证。**

2023 年 Claude 100K → NIAH 评测出现 → 发现许多模型"注水"。

2024 年 1M 模型发布 → RULER/BABILong 出现 → 暴露 10-20% 真实利用率。

2025-2026 年 10M 模型 → NoLiMa 出现 → 宣称和有效的差距仍以数量级计。

这个 pattern 不是巧合。**评测方法总是落后于模型发布，给了厂商"先发布再被验证"的空间。** 但 NoLiMa 的出现说明市场正在成熟——用户不再盲目相信宣称数字。

我的判断是：**2027-2028 年，"有效上下文"将取代"宣称上下文"成为核心竞争指标。** NoLiMa 或类似基准将成为模型的"标配测试"，没有通过的模型将面临信任危机。

5.3 三条暗线：系统、成本和评估

长上下文技术从来不只是一个"算法问题"。

系统能力是常被低估的暗线。Google 能在 2024 年率先推出 1M 模型，不是因为算法比 OpenAI 高明，而是因为 TPU 集群 + 自研分布式系统的工程积累。DeepSeek 用 MLA 把 KV Cache 压缩 93%，也是系统级优化（存储→计算 trade-off）的胜利。FlashAttention 更是纯粹的"系统工程"。

成本是另一条暗线。2023 年的 100K 上下文调用成本是 $5–10/次，2026 年降到了 $0.5–5/次（取决于模型和缓存策略）。DeepSeek 把 1M 上下文的价格打到接近零——这正在改变长上下文的产品化可能性。

评估从"事后验证"变成了"推动力"。RULER 和 BABILong 发现的问题，直接影响了 2025–2026 年的模型训练策略。NoLiMa 的"语义 needle"概念可能催生新一代上下文扩展方法。

5.4 Dilution: 长上下文的核心矛盾

长上下文领域有一个深层矛盾，我称之为 **Dilution Problem（稀释问题）**——token 越多，注意力越分散，相关性越弱。

这不是架构问题，是数学问题。在 N 个 token 上算 softmax，每个 token 分到的注意力比例大致是 1/N。N=4K 时这个问题不严重，N=1M 时 token 的相关性被稀释到接近零。

现有方案的应对：

- **Local + Global 混合**（Longformer/Mistral）：局部窗口保精度，全局 token 保覆盖
- **Attention Sink**（StreamingLLM）：把"多余的注意力"倾倒给初始 token
- **KV Cache 剪枝**（H2O/Keyformer）：只保留"重要"token，抛弃"不重要"的
- **iRoPE**（Llama 4 Scout）：交替使用有/无位置编码层，牺牲精度换覆盖

但所有这些方案都只是缓解，没有根治。**只要模型需要在 N 个 token 间分配注意力，稀释问题就存在。**

我的判断：**未来 2–3 年，长上下文技术的最大突破将来自"注意力稀释"的数学解法**——可能是一种新的归一化方式（替代 softmax），或者一种"分阶段注意力"架构（先粗筛再细读），而不是在现有框架内打补丁。

5.5 竞品的发展路径对比

如果把主要模型厂商放到时间线上对比：

- **Google**：从 2024 年的 Gemini 1.5（1M）到 2025 年的 2.5 Pro（1M+），一直领跑上下文竞赛。优势在基础设施（TPU）、数据（YouTube/Books corpus）、和在搜索中积累的长文本处理经验。劣势在商业化和开发者生态——API 不如 OpenAI 常用。
- **OpenAI**：从 GPT-4 Turbo（128K）到 GPT-5.5（1M），走的是"稳妥扩长"路线——每次只比对手多一点，但把质量做到最优。但在 NoLiMa 上有效上下文仅 16K，说明"稳妥扩长"不一定带来有效增长。
- **Anthropic**：2023 年的 Claude 100K/200K 是长上下文最早的领跑者。但在 2024 年 Gemini 跨入 1M 后，Anthropic 没有立即跟进（Claude 3.5 仍为 200K），直到 2025–2026 年的 Claude 4 系列才追上。策略差异反映公司文化：Anthropic 更谨慎，更倾向在安全评估确认后再放量。
- **DeepSeek**：走的是"成本效率"路线。MLA 把 KV Cache 压缩 93%+，根本性地降低了长上下文的成本。API 定价是行业最低水平之一。2025–2026 年的增长主要靠性价比驱动。
- **Meta (Llama)**：开源路线，上下文扩展靠社区贡献（YaRN、NTK 都来自社区而非 Meta）。Llama 4 Scout 的 10M 是行业最高，但有效利用率也是最低的。

5.6 三个未来剧本

剧本一：渐进收敛（概率 60%）

上下文长度稳定在 1M–2M 有效范围，不再疯长。评测标准升级（NoLiMa 常态化），RoPE + MLA 成为标准技术栈。长上下文和 RAG 融合为统一的"Context Engineering"框架。**这是最可能的路径——技术没有根本性的新突破，只是现有方案不断优化和收敛。**

剧本二：注意力瓶颈打破（概率 25%）

一种新的注意力机制出现——可能是分阶段注意力（先粗筛再细读）、或者新的归一化方式替代 softmax。有效上下文从 1M 跳到 100M+，"Lost in the Middle" 被大幅缓解。**需要数学突破，不是工程优化——不确定性强。** 如果这个突破来自学术界而非大厂，将对行业格局产生深远影响。

剧本三：上下文竞赛泡沫破裂（概率 15%）

NoLiMa 类评测成为行业标准后，用户发现所有模型的"有效上下文"都远低于宣称。信任危机+回归理性：上下文长度不再是模型定价的核心参数。RAG 全面回归，长上下文被视为"锦上添花"而非核心竞争力。**概率低，但如果在 2027 年 NoLiMa 升级版暴露出更大差距，不排除这个剧本。**

5.7 长上下文：是"所有人都需要"还是"锦上添花"?

一个问题值得思考：我们真的需要 1M+ 上下文吗?

从产品使用数据看，绝大多数实际场景在 32K–128K 范围内。代码理解、论文分析、法律合同——这些"长文本场景"通常也在 100K 以内（《三体》三部曲约 200 万字 ≈ 2M token，但谁会一次性读三本？）。

长上下文的核心用户是 **Agent 和复杂推理系统**——它们需要在一次调用中看完整个代码库、整个对话历史、多份交叉引用的文档。这是真正的"刚需场景"，但也是极低频场景。

所以我的判断是：**百万级上下文是"保险"而非"日常"。** 它决定了一个模型的上限能力，但普通用户的日常体验更多被"32K–128K 范围内的准确率"决定。厂商在展示 1M 上下文的同时，更应该关注 100K 内的稳定性——这才是 80% 用户的核心体验。

5.6 2026 年 4–5 月加速：FlashAttention–4 与 NSA

FlashAttention–4（Tri Dao, 2026.03） 是 2026 年长上下文推理基础设施最重要的事件。FA4 专门针对 Blackwell GPU 的不对称硬件架构做了算法–内核联合设计——H100 上用 FA3 足够，B200 上必须用 FA4 才能释放有效算力。核心创新是将 attention 计算流水线化，矩阵乘法级的速度，实测将 B200 上原本浪费的 60% 算力利用率提升到 71%。

DeepSeek NSA（Native Sparse Attention, 2025 年末 → 2026 年全面落地在 V4） 是另一个影响深远的方向。NSA 不等同于传统的"事后稀疏"——而是在训练阶段就内建稀疏性。V4 的 CSA（Compressed Sparse Attention）+ HCA（Hierarchical Context Attention）混合方案，让 1M 上下文的实际可用性远超 Llama 4 Scout（后者 10M 宣称但有效仅 ~1K）。V4 的输出长度达到 384K，是行业平均的 3–6 倍——"能读"和"能写"是两个不同的长上下文能力维度。

LongRoPE2（Microsoft, 2026） 实现了近无损的上下文窗口扩展——在已有 RoPE 模型上做 post–training 即可显著延长上下文，不需要从头训练。这降低了长上下文能力获取的门槛。

六、信息来源

核心论文与文章

论文/文章	来源	访问时间
Attention Is All You Need	arxiv.org/abs/1706.03762	2026-04-30
Sparse Transformer	arxiv.org/abs/1904.10509	2026-04-30
Multi-Query Attention	arxiv.org/abs/1911.02150	2026-04-30
Reformer	arxiv.org/abs/2001.04451	2026-04-30
Longformer	arxiv.org/abs/2004.05150	2026-04-30
BigBird	arxiv.org/abs/2007.14062	2026-04-30
Long Range Arena (LRA)	arxiv.org/abs/2011.04006	2026-04-30
RoFormer (RoPE)	arxiv.org/abs/2104.09864	2026-04-30
ALiBi	arxiv.org/abs/2108.12409	2026-04-30
FlashAttention	arxiv.org/abs/2205.14135	2026-04-30
Lost in the Middle	arxiv.org/abs/2307.03172	2026-04-30
Position Interpolation (PI)	arxiv.org/abs/2306.15595	2026-04-30
FlashAttention-2	arxiv.org/abs/2307.08691	2026-04-30
Code Llama	arxiv.org/abs/2308.12950	2026-04-30
YaRN	arxiv.org/abs/2309.00071	2026-04-30
GQA	arxiv.org/abs/2305.13245	2026-04-30
DeepSpeed Ulysses	arxiv.org/abs/2309.14509	2026-04-30
Ring Attention	arxiv.org/abs/2310.01889	2026-04-30
StreamingLLM	arxiv.org/abs/2309.17453	2026-04-30
H2O	arxiv.org/abs/2306.12022	2026-04-30
SGLang	arxiv.org/abs/2312.07104	2026-04-30
CLongEval	arxiv.org/abs/2403.03514	2026-04-30
Gemini 1.5	arxiv.org/abs/2403.05530	2026-04-30
RULER	arxiv.org/abs/2404.06654	2026-04-30
DeepSeek-V2 (MLA)	arxiv.org/abs/2405.04434	2026-04-30

BABILong	arxiv.org/abs/2406.10149	2026-04-30
LLaMA 3	arxiv.org/abs/2407.21783	2026-04-30
KVQuant	arxiv.org/abs/2401.18079	2026-04-30
YaRN (original)	arxiv.org/abs/2309.00071	2026-04-30

产品与技术来源

来源	URL	访问时间
Google Cloud — Gemini Long Context	ai.google.dev/gemini-api/docs/long-context	2026-04-30
Anthropic — Claude Prompt Caching	docs.anthropic.com/en/docs/build-with-claude/prompt-caching	2026-04-30
Codingscape — Top LLMs 2026	codingscape.com/blog/most-powerful-llms-2026	2026-04-30
WhatLLM — Largest Context	whatllm.org/largest-context-window	2026-04-30
BenchLM — Context Window Comparison 2026	benchlm.ai/blog/posts/context-window-comparison-2026	2026-04-30
Atlan — Context Limitations 2026	atlan.com/learn-with-atlan/llm-context-window-limitations-2026	2026-04-30
Openlayer — Needle Test Guide Jan 2026	openlayer.com/blog/post/needle-in-a-haystack-ai-testing	2026-04-30
Mem0 — LLM API Cost Breakdown 2026	mem0.ai/blog/llm-api-cost-breakdown	2026-04-30
Sebastian Raschka — MLA	sebastianraschka.com/blog/2025/multi-head-latent-attention	2026-04-30
Lior Sinai — DeepSeek MLA	liorsinai.github.io/blog/2025/02/22/mla	2026-04-30
Towards Data Science — RoPE Explained	towardsdatascience.com (Jan 2026)	2026-04-30
mmntm.net — NoLiMa 评测	mmntm.net	2026-04-30
腾讯网 — DeepSeek V4 Pro 报道	腾讯网	2026-04-30
Google Cloud — Llama 4 博客	blog.google	2026-04-30

本文是横纵分析系列的第 10 篇报告。研究方法论参考了由数字生命卡兹克（Khazix）提出的横纵分析法，融合语言学历时-共时分析、社会科学纵向-横截面研究设计、以及竞争战略分析的核心思想。